Akash Kumar Saw
AIML (03)

Experiment No. : 05

Title : Write a program to implement the Logistic Regression for a sample training data set stored as a .CSV file. Compute the accuracy of the classifier.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

sample = pd.DataFrame({
    "Hours": [1, 2, 2.5, 3, 3.5, 4, 5, 5.5, 6, 7, 8, 9],
    "Pass":  [0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1]
})

sample.to_csv("logistic_data.csv", index=False)

data = pd.read_csv("logistic_data.csv")

X = data[["Hours"]]
y = data["Pass"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.33, random_state=42, stratify=y
)

model = LogisticRegression()
model.fit(X_train, y_train)

predictions = model.predict(X_test)

print("Predictions:", predictions)
print("Accuracy:", accuracy_score(y_test, predictions))
```

```
Predictions: [0 1 1 1]
Accuracy: 1.0
```

Akash Kumar Saw
AIML (03)

Experiment No. : 06

Title : Write a program to implement the naive Bayesian classifier for a sample training data set stored as a .CSV file. Compute the accuracy of the classifier, considering few test data sets.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score

sample = pd.DataFrame({
    "Feature1": [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12],
    "Feature2": [2, 1, 3, 2, 4, 5, 6, 5, 7, 8, 9, 10],
    "Class":    [0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1]
})

sample.to_csv("naive_bayes_data.csv", index=False)

data = pd.read_csv("naive_bayes_data.csv")

X = data[["Feature1", "Feature2"]]
y = data["Class"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.33, random_state=42, stratify=y
)

model = GaussianNB()
model.fit(X_train, y_train)

predictions = model.predict(X_test)

print("Predictions:", predictions)
print("Actual Values:", y_test.tolist())
print("Accuracy:", accuracy_score(y_test, predictions))

# Test a few new data samples
test_data = pd.DataFrame({
    "Feature1": [2, 5, 8, 11],
    "Feature2": [1, 4, 5, 9]
})

test_predictions = model.predict(test_data)
```

```
print("New Test Data Predictions:", test_predictions)
```

```
Predictions: [0 1 0 1]
Actual Values: [0, 1, 0, 1]
Accuracy: 1.0
New Test Data Predictions: [0 0 1 1]
```

Akash Kumar Saw
AIML (03)

Experiment No. : 07

Title : Write a program to implement k-Nearest Neighbor algorithm to classify the Iris data set. Print both correct and wrong predictions.

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier

iris = load_iris()

X = iris.data
y = iris.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.30, random_state=42
)

model = KNeighborsClassifier(n_neighbors=3)
model.fit(X_train, y_train)

predictions = model.predict(X_test)

for actual, predicted in zip(y_test, predictions):
    status = "Correct" if actual == predicted else "Wrong"

    print(
        "Actual:", iris.target_names[actual],
        "Predicted:", iris.target_names[predicted],
        "-", status
    )
```

```
Actual: versicolor Predicted: versicolor - Correct
Actual: setosa Predicted: setosa - Correct
```

```
Actual: virginica Predicted: virginica - Correct
Actual: versicolor Predicted: versicolor - Correct
Actual: versicolor Predicted: versicolor - Correct
Actual: setosa Predicted: setosa - Correct
Actual: versicolor Predicted: versicolor - Correct
Actual: virginica Predicted: virginica - Correct
Actual: versicolor Predicted: versicolor - Correct
Actual: versicolor Predicted: versicolor - Correct
Actual: virginica Predicted: virginica - Correct
Actual: setosa Predicted: setosa - Correct
Actual: setosa Predicted: setosa - Correct
Actual: setosa Predicted: setosa - Correct
Actual: setosa Predicted: setosa - Correct
Actual: versicolor Predicted: versicolor - Correct
Actual: virginica Predicted: virginica - Correct
Actual: versicolor Predicted: versicolor - Correct
Actual: versicolor Predicted: versicolor - Correct
Actual: virginica Predicted: virginica - Correct
Actual: setosa Predicted: setosa - Correct
Actual: virginica Predicted: virginica - Correct
Actual: setosa Predicted: setosa - Correct
Actual: virginica Predicted: virginica - Correct
Actual: virginica Predicted: virginica - Correct
Actual: virginica Predicted: virginica - Correct
Actual: virginica Predicted: virginica - Correct
Actual: virginica Predicted: virginica - Correct
Actual: setosa Predicted: setosa - Correct
Actual: setosa Predicted: setosa - Correct
Actual: setosa Predicted: setosa - Correct
Actual: setosa Predicted: setosa - Correct
Actual: versicolor Predicted: versicolor - Correct
Actual: setosa Predicted: setosa - Correct
Actual: setosa Predicted: setosa - Correct
Actual: virginica Predicted: virginica - Correct
Actual: versicolor Predicted: versicolor - Correct
Actual: setosa Predicted: setosa - Correct
Actual: setosa Predicted: setosa - Correct
Actual: setosa Predicted: setosa - Correct
Actual: virginica Predicted: virginica - Correct
Actual: versicolor Predicted: versicolor - Correct
Actual: versicolor Predicted: versicolor - Correct
Actual: setosa Predicted: setosa - Correct
Actual: setosa Predicted: setosa - Correct
```

Akash Kumar Saw
AIML (03)

Experiment No. : 08

Title : Write a program to demonstrate the working of the decision tree based ID3

algorithm. Use an appropriate data set for building the decision tree and apply this knowledge to classify a new sample.

```
import pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder
from sklearn.pipeline import Pipeline
from sklearn.tree import DecisionTreeClassifier

# Play Tennis dataset
X = pd.DataFrame({
    "Outlook": [
        "Sunny", "Sunny", "Overcast", "Rain",
        "Rain", "Rain", "Overcast", "Sunny"
    ],
    "Temperature": [
        "Hot", "Hot", "Hot", "Mild",
        "Cool", "Cool", "Cool", "Mild"
    ],
    "Humidity": [
        "High", "High", "High", "High",
        "Normal", "Normal", "Normal", "High"
    ],
    "Wind": [
        "Weak", "Strong", "Weak", "Weak",
        "Weak", "Strong", "Strong", "Weak"
    ]
})

y = ["No", "No", "Yes", "Yes", "Yes", "No", "Yes", "No"]

# Convert categorical features into numerical values
preprocessor = ColumnTransformer([
    ("cat", OneHotEncoder(handle_unknown="ignore"), X.columns)
])

# Build the decision tree using entropy
id3 = Pipeline([
    ("prep", preprocessor),
    ("tree", DecisionTreeClassifier(
        criterion="entropy",
        random_state=42
    ))
])

# Train the model
id3.fit(X, y)

# Classify a new sample
```

```
new_sample = pd.DataFrame([{
    "Outlook": "Overcast",
    "Temperature": "Mild",
    "Humidity": "High",
    "Wind": "Weak"
}])

print("Prediction:", id3.predict(new_sample)[0])
```

```
Prediction: Yes
```

Akash Kumar Saw
AIML (03)

Experiment No. : 09

Title : Write a program to demonstrate the working of the decision tree based CART algorithm. Use an appropriate data set for building the decision tree and apply this knowledge to classify a new sample.

```
import pandas as pd
from sklearn.tree import DecisionTreeClassifier

# Sample dataset
X = pd.DataFrame({
    "Age": [22, 25, 28, 32, 35, 40, 45, 50],
    "Income": [25000, 30000, 35000, 45000, 50000, 65000, 70000, 85000]
})

y = [0, 0, 0, 1, 1, 1, 1, 1]

# Create the CART model using Gini impurity
model = DecisionTreeClassifier(
    criterion="gini",
    random_state=42
)

# Train the model
model.fit(X, y)

# Classify a new sample
new_sample = pd.DataFrame({
    "Age": [30],
    "Income": [42000]
```

```
})

prediction = model.predict(new_sample)[0]

print("Predicted Class:", prediction)
```

```
Predicted Class: 0
```

Akash Kumar Saw
AIML (03)

Experiment No. : 10 Title : Implement a single neural network and test for different logic gates.

```
import numpy as np
from sklearn.neural_network import MLPClassifier

X = np.array([
    [0, 0],
    [0, 1],
    [1, 0],
    [1, 1]
])

gates = {
    "AND": np.array([0, 0, 0, 1]),
    "OR":  np.array([0, 1, 1, 1]),
    "XOR": np.array([0, 1, 1, 0])
}

for gate, y in gates.items():
    model = MLPClassifier(
        hidden_layer_sizes=(4,),
        solver="lbfgs",
        max_iter=5000,
        random_state=42
    )

    model.fit(X, y)
    predictions = model.predict(X)

    print("\n", gate, "Gate")
    for inp, out in zip(X, predictions):
```

```
            print(inp, "->", out)
```

```
 AND Gate
[0 0] -> 0
[0 1] -> 0
[1 0] -> 0
[1 1] -> 1

 OR Gate
[0 0] -> 0
[0 1] -> 1
[1 0] -> 1
[1 1] -> 1

 XOR Gate
[0 0] -> 0
[0 1] -> 0
[1 0] -> 1
[1 1] -> 0
```